

SEMINÁRIO UNIVERSITÁRIO • 2026

Serenity BDD

Testes que viram documentação.

João Oliveira
Segurança e Teste de Software • IDP



O problema dos testes automatizados

Testes automatizados ajudam muito, mas costumam ter três problemas:

01

Ilegíveis

O código do teste só faz sentido para quem programou. O cliente e o analista não conseguem ler.

02

Sem evidência

O teste passou, mas não fica registrado o que foi testado nem como a tela estava.

03

Repetitivos

Quando a suíte cresce, o mesmo código de clicar e digitar se repete em vários testes.

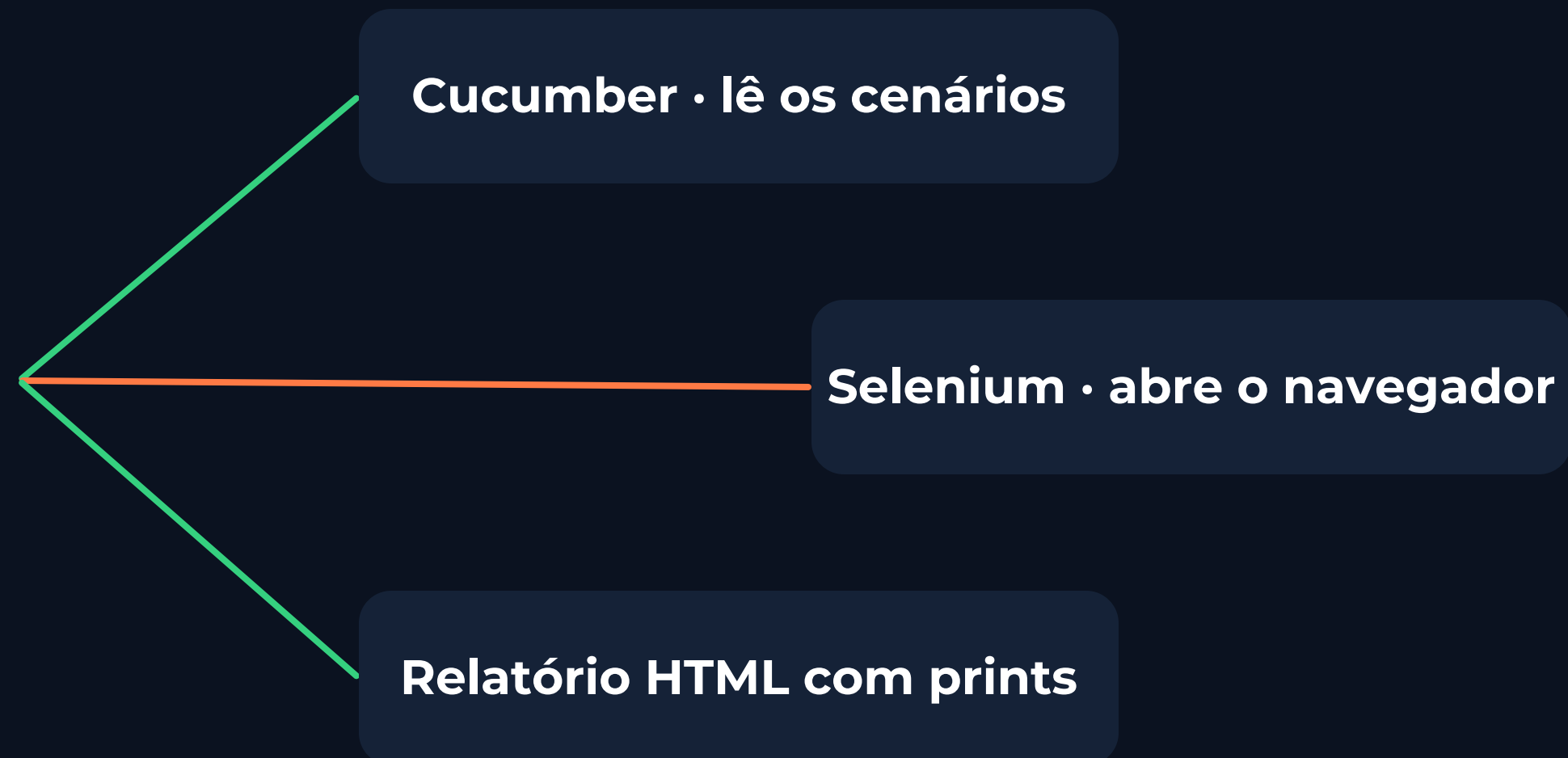
Resultado: os testes existem, mas ninguém confia neles. O Serenity BDD ataca esses três pontos.

O que é o Serenity BDD

Uma biblioteca Java que transforma os testes automatizados em documentação viva: um relatório com cada passo e um print da tela.

Serenity

fica por cima de outras ferramentas



- Criado por John Ferguson Smart, autor do livro BDD in Action (que está na nossa bibliografia).
- Gratuito e de código aberto. Começou com o nome Thucydides e virou Serenity em 2014.

Onde fica na pirâmide de testes

O Serenity fica no topo (testes de aceitação) e também pode testar APIs.

Aceitação · Serenity

Simula o usuário na tela. É o mais lento: use nos fluxos importantes.

Serviço / API · Serenity + REST Assured

Testa a API direto, sem abrir o navegador.

Unidade · JUnit, Mockito

Base: testa cada classe sozinha. Aqui não se usa o Serenity, e sim JUnit e Mockito.

Principais recursos

O que o Serenity traz além de só controlar o navegador:

01

Documentação viva

Relatório HTML com cada cenário, cada passo e print da tela.

02

Cenários em português

Frases com Dado, Quando, Então que viram teste de verdade.

03

Padrão Screenplay

Um ator faz tarefas, como numa história. Código fácil de ler.

04

Asserções legíveis

Ensure.that(...) aparece como uma frase no relatório.

05

Prints automáticos

Print a cada passo ou só quando o teste falha. Você escolhe.

06

Web, API e mobile

Selenium ou Playwright, REST Assured e Appium no mesmo projeto.

O padrão Screenplay

O teste vira uma história: um ator (a Ana) faz tarefas na tela. O código fica em três pastas:



Pasta: telas/
Ex.: TelaDeLogin

Telas

Dizem ONDE ficam os campos e botões da página.



Pasta: tarefas/
Ex.: FazerLogin

Tarefas

Dizem O QUE o usuário faz: entrar, adicionar, concluir.



Pasta: passos/
Ex.: PassosDeLogin

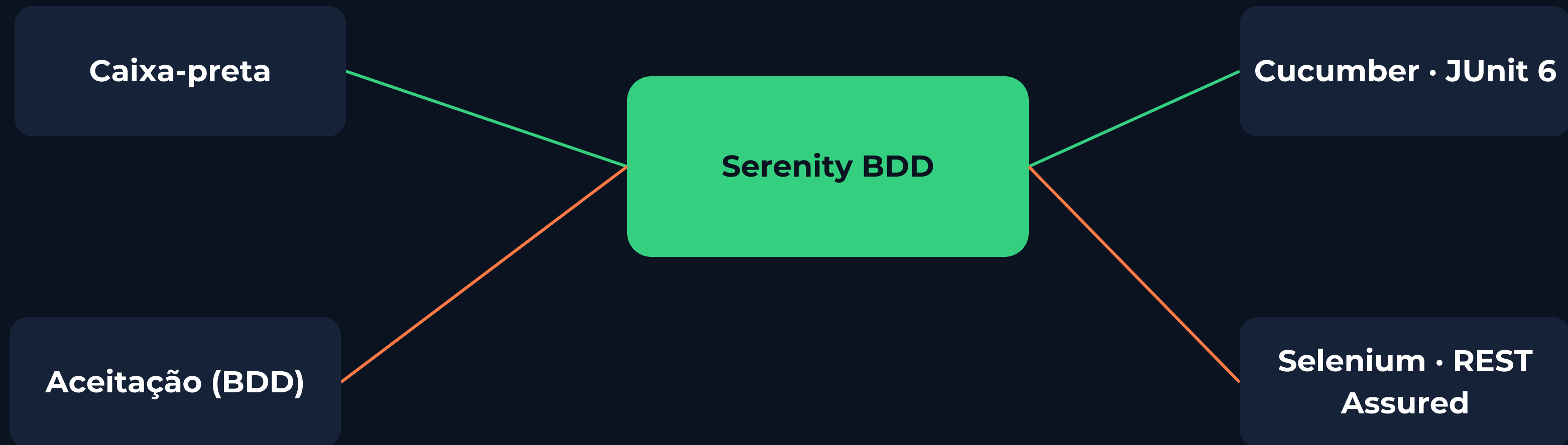
Passos

Ligam cada frase do cenário em português ao código Java.

Vantagem: se a tela mudar, eu conserto em um lugar só, e todos os testes continuam funcionando.

Tipos de teste e integrações

Como o Serenity testa (esquerda) e com o que ele se conecta (direita):



Caixa-preta: testa só pelo que o usuário vê, sem olhar o código da aplicação.

Aceitação: confere se o sistema faz o que o cliente pediu, cenário por cenário.

Também se conecta com Maven, Gradle, Jenkins, GitHub Actions e Jira.

Demonstração ao vivo

LOGIN + LISTA DE TAREFAS • 6 CENÁRIOS

```
# login.feature (o que o cliente lê)
Cenário: Login com usuário e senha corretos
  Dado que Ana está na tela de login
  Quando ela entra com usuário "aluno" e senha "idp123"
  Então ela deve ver a mensagem de boas-vindas "Olá, aluno!"

// PassosDeLogin.java (o que o Java faz)
@Quando("{actor} entra com usuário {string} e senha {string}")
public void entraCom(Actor ator, String usuario, String senha) {
    ator.attemptsTo(FazerLogin.com(usuario, senha));
}
```

1. Escrever
0 cenário em português, no arquivo .feature.
2. Ligar
Cada frase vira um método Java.
3. Rodar
mvn clean verify roda os 6 cenários.
4. Ver o relatório
target/site/serenity/index.html, com prints de cada passo.

Repare: o cenário é português puro. O Serenity liga cada frase ao código e registra tudo no relatório.

Frameworks similares

Todos fazem testes de aceitação legíveis. A diferença está no relatório e na organização do código.

Ferramenta	Camada	Linguagens
Cucumber puro — só lê os cenários	Aceitação	Java · JS · Ruby · outras
JBehave — o BDD mais antigo em Java	Aceitação	Java
Robot Framework — palavras-chave	Aceitação	Python
Serenity BDD — BDD + relatório vivo	Aceitação + API	Java (e Serenity/JS para JavaScript)

Também existem o Gauge (testes em Markdown), o Karate (API sem Java) e o Allure (só relatórios).

Serenity × Cucumber × Robot Framework

Três jeitos de fazer testes de aceitação que qualquer pessoa consegue ler:

Serenity BDD

Relatório vivo com prints
Screenplay organiza o código
Web, API e mobile
Só Java (JS é outro projeto)
Mais configuração

→ Melhor se o relatório importa

Cucumber puro

Lê os cenários em Gherkin
Várias linguagens
Relatório simples
Você organiza o código
Mais leve

→ Bom para começar com BDD

Robot Framework

Testes por palavras-chave
Feito em Python
Relatório próprio (log.html)
Fácil para quem não programa
Comunidade grande

→ Bom para times de QA sem Java

Resumo: para Java, o Serenity é o mais completo. O preço é ter mais coisa para aprender e configurar.

Vantagens e desvantagens

VANTAGENS

Relatório vivo, com prints de cada passo

Cenários em português que todos entendem

Screenplay: código reproveitável

Web, API e mobile no mesmo projeto

Maduro: mais de 10 anos, versão 5 atual

DESVANTAGENS

Curva de aprendizado: muita coisa nova

Mais configuração que Selenium sozinho

Print de cada passo deixa o teste lento

Comunidade menor que a do Selenium

Só Java (para JS existe o Serenity/JS)

Resumo: vale a pena quando o relatório e a organização importam mais que a simplicidade.

Casos de sucesso

Não existe uma lista oficial de empresas, mas há exemplos públicos:

01



BDD in Action

Livro da Manning (2023) escrito pelos criadores do Serenity. Está na bibliografia da disciplina.

02



Xebia

Consultoria de software que testou o Serenity e chamou o relatório de “o mais bonito” que já tinha visto.

03



Nesta disciplina

A aula z03 do curso usa Serenity, Cucumber e Screenplay para testar o TodoMVC.

Ou seja: é uma ferramenta madura, ensinada em livro e usada há mais de 10 anos em projetos Java.

Conclusão

ADOTE QUANDO

O projeto já usa BDD (Dado, Quando, Então)

É preciso mostrar evidência dos testes

A suíte de aceitação em Java é grande

Gente que não programa lê os testes

EVITE QUANDO

O projeto é pequeno ou é um protótipo

O time não conhece Java

Só precisa de testes de unidade

O time é de JavaScript (veja o Serenity/JS)

Testes que qualquer pessoa consegue ler.

Ou seja: use o Serenity quando o relatório e os cenários em português trazem valor para o time.

FIM DO CENÁRIO

Obrigado! Perguntas?

João Oliveira · IDP · 2026

Código e README: [workshops/01-test/submissions/joao-oliveira](https://github.com/serenity-bdd/workshops/01-test/submissions/joao-oliveira)

Documentação oficial: serenity-bdd.github.io

